

Actividad 8: Algoritmos Voraces (Greedy)

Materia: Estructura de Datos y Algoritmos

Entregable: PDF + Repositorio GitHub

Objetivo

Aplicar la estrategia de algoritmos voraces (*Greedy*) para resolver problemas de optimización, además de analizar formalmente el comportamiento, correctitud y la complejidad asintótica de los algoritmos implementados.

Repositorio GitHub

Enlace al repositorio: <https://github.com/ZapanaDiego/Analisis-y-Disenio-de-Algoritmos/tree/main/AlgoritmosVoracesActividad08>

Enlace al repositorio: [git@github.com:ZapanaDiego/Analisis-y-Disenio-de-Algoritmos.git](https://github.com/ZapanaDiego/Analisis-y-Disenio-de-Algoritmos.git)

1 Problema 1: Mochila Fraccional

1.1 Descripción y Fundamento Teórico

El problema de la Mochila Fraccional plantea la necesidad de maximizar el valor total de un conjunto de objetos cargados en una mochila con una capacidad de peso limitada (W). Cada objeto i posee un valor v_i y un peso w_i . A diferencia del problema clásico de la mochila 0/1, el dominio de este problema **permite el fraccionamiento**; es decir, se puede tomar una porción $x_i \in [0, 1]$ de cualquier objeto.

1.2 Estrategia Greedy y Demostración de Optimalidad

La estrategia voraz consiste en evaluar la **densidad de valor** de cada objeto, definida como la proporción $d_i = \frac{v_i}{w_i}$ (valor aportado por cada unidad de peso).

1. Se calcula la densidad d_i para los N objetos.
2. Se ordena el conjunto de objetos en orden descendente respecto a su densidad.
3. Se itera sobre los objetos ordenados: si la capacidad restante lo permite, se toma el objeto íntegramente ($x_i = 1$). Si el objeto excede la capacidad actual, se toma la fracción exacta ($x_i < 1$) que agote el espacio de la mochila, y el algoritmo termina.

Justificación formal de optimalidad: La propiedad de elección greedy garantiza una solución óptima global. Supongamos por contradicción que existe una solución óptima alternativa que *no* prioriza los objetos por mayor d_i . En dicha solución, existe un objeto con densidad d_x excluido o fraccionado, mientras que un objeto con densidad d_y (donde $d_y < d_x$) fue incluido. Dado que se permiten fracciones, podríamos intercambiar una unidad de peso del objeto y por una unidad del objeto x . Este intercambio aumentaría el valor total de la mochila (ya que $d_x > d_y$).

sin alterar el peso total. Esto contradice la premisa de que la solución alternativa era óptima. Por inducción, la estrategia greedy es matemáticamente exacta.

1.3 Análisis de Complejidad Asintótica (Big-O)

- **Complejidad Temporal:** $O(N \log N)$

El algoritmo se compone de tres fases: 1) Calcular el ratio d_i toma $O(N)$. 2) Ordenar los objetos (típicamente mediante Timsort o Mergesort en Java `Arrays.sort`) exige, en el peor caso, $O(N \log N)$ comparaciones. 3) El bucle iterativo para llenar la mochila procesa cada elemento a lo sumo una vez, requiriendo $O(N)$. Por la regla de la suma en el análisis asintótico, el término dominante dicta la complejidad temporal total: $O(N \log N) + O(N) = O(N \log N)$.

- **Complejidad Espacial:** $O(N)$

Se requiere memoria $O(N)$ para almacenar el arreglo de los N objetos instanciados en memoria. Adicionalmente, el algoritmo de ordenamiento subyacente (Mergesort/Timsort) en Java de objetos requiere $O(N)$ espacio auxiliar en el peor caso. Por lo tanto, el espacio adicional requerido escala linealmente.

1.4 Código Fuente en Java

```
1 import java.util.Arrays;
2 import java.util.Comparator;
3
4
5 public class MochilaFraccional {
6
7     static class Objeto {
8         String nombre;
9         double valor;
10        double peso;
11        double ratio;
12
13        Objeto(String nombre, double valor, double peso) {
14            this.nombre = nombre;
15            this.valor = valor;
16            this.peso = peso;
17            this.ratio = valor / peso;
18        }
19    }
20
21    static void print(String mensaje) {
22        System.out.println(mensaje);
23    }
24
25    static double resolverMochila(Objeto[] objetos, double capacidad) {
26
27        Arrays.sort(objetos, Comparator.comparingDouble((Objeto o) -> o.ratio).reversed());
28
29        double valorTotal = 0.0;
30        double pesoRestante = capacidad;
31
32        print("== Mochila Fraccional ==");
33        print(String.format("Capacidad maxima: %.0f", capacidad));
34        print("");
35        print("Objetos seleccionados:");
36
37        for (Objeto obj : objetos) {
```

```

38         if (pesoRestante <= 0) break; // mochila llena
39
40         if (obj.peso <= pesoRestante) {
41             // Tomar el objeto completo
42             valorTotal += obj.valor;
43             pesoRestante -= obj.peso;
44             print(String.format("  %s completo (valor: %.1f, peso: %.1f, ratio: %.2f
)",
45
46                 obj.nombre, obj.valor, obj.peso, obj.ratio));
47         } else {
48             // Tomar solo la fraccion que cabe
49             double fraccion = pesoRestante / obj.peso;
50             valorTotal += fraccion * obj.valor;
51             print(String.format("  %.0f/%.0f Parte del objeto %s (valor parcial: %.2
f, ratio: %.2f)",
52
53                 pesoRestante, obj.peso, obj.nombre, fraccion * obj.valor, obj.
54                 ratio));
55             pesoRestante = 0;
56         }
57     }
58     print("");
59     print(String.format("Valor total obtenido: %.2f", valorTotal));
60     return valorTotal;
61 }
62
63 public static void main(String[] args) {
64
65     Objeto[] objetos1 = {
66         new Objeto("A", 60, 10),
67         new Objeto("B", 100, 20),
68         new Objeto("C", 120, 30)
69     };
70     resolverMochila(objetos1, 50);
71
72     print("");
73     print("-----");
74     print("");
75
76     // Ejemplo 2
77     Objeto[] objetos2 = {
78         new Objeto("A", 80, 20),
79         new Objeto("B", 100, 10),
80         new Objeto("C", 120, 30)
81     };
82     resolverMochila(objetos2, 25);
83 }

```

Listing 1: Implementación algorítmica de la Mochila Fraccional

1.5 Ejemplos de Ejecución y Trazado

Ejemplo 1: Capacidad de la mochila = 50. Objetos iniciales: A(60, 10), B(100, 20), C(120, 30).

Ejemplo 2: Capacidad de la mochila = 25. Objetos iniciales: A(80, 20), B(100, 10), C(120, 30).

Paso	Objeto Seleccionado	Ratio (v/w)	Peso Utilizado	Beneficio Acumulado
1	Objeto A (Completo)	6.0	10 de 50	60.00
2	Objeto B (Completo)	5.0	20 de 40	160.00
3	Objeto C (Fracción 20/30)	4.0	20 de 20	240.00

Cuadro 1: Trazado del Ejemplo 1. Resultado global: **240.00**

Paso	Objeto Seleccionado	Ratio (v/w)	Peso Utilizado	Beneficio Acumulado
1	Objeto B (Completo)	10.0	10 de 25	100.00
2	Objeto A (Fracción 15/20)	4.0	15 de 15	160.00

Cuadro 2: Trazado del Ejemplo 2. Resultado global: **160.00**

2 Problema 2: Cobertura de Antenas

2.1 Descripción y Fundamento Teórico

El objetivo es instalar la cantidad mínima de antenas WiFi para cubrir un conjunto de N casas ubicadas a lo largo de un eje unidimensional (carretera). Una antena posicionada en p proporciona cobertura en un radio R , abarcando el intervalo $[p - R, p + R]$.

2.2 Estrategia Greedy y Demostración de Optimalidad

La estrategia (conocida algorítmicamente como *Interval Covering*) se basa en estirar la cobertura de cada antena lo más a la derecha posible, cubriendo la casa desprotegida más a la izquierda en el extremo absoluto del rango de la antena.

1. Se ordenan las posiciones de las casas de manera ascendente.
2. Sea x la posición de la primera casa sin cobertura. Para asegurar que x reciba señal, la antena debe estar en algún punto $p \in [x - R, x + R]$.
3. **Decisión Greedy:** Elegimos colocar la antena exactamente en $p = x + R$.
4. **Justificación formal:** Colocar la antena en $x + R$ asegura que la casa en x queda cubierta por el borde izquierdo del intervalo (ya que $(x + R) - R = x$). A su vez, el borde derecho del alcance se extiende hasta $(x + R) + R = x + 2R$. Cualquier posición $p < x + R$ cubriría menos espacio a la derecha, potencialmente requiriendo más antenas para futuras casas. Por tanto, esta decisión maximiza el alcance útil sin violar las restricciones, resultando en un mínimo global de antenas.

2.3 Análisis de Complejidad Asintótica (Big-O)

- **Complejidad Temporal:** $O(N \log N)$

Ordenar el arreglo de posiciones de tamaño N toma $O(N \log N)$ mediante Dual-Pivot Quicksort (`Arrays.sort` para primitivos en Java). Posteriormente, existe un bucle `while` exterior y un `while` interior para iterar sobre el arreglo. A primera vista, los bucles anidados podrían sugerir una complejidad cuadrática, sin embargo, el puntero del arreglo `i` es un contador monótono estricto. Cada casa se verifica exactamente una vez, adelantando el puntero continuamente. El recorrido completo es estrictamente $O(N)$. Asintóticamente: $O(N \log N) + O(N) = \mathbf{O(N \log N)}$.

- **Complejidad Espacial:** $O(N)$ o $O(1)$ **auxiliar**

Si se ignora el espacio requerido para almacenar los datos de entrada y la salida de tamaño a lo sumo N , el algoritmo iterativo opera *in-place* utilizando variables primitivas de control que toman espacio $O(1)$. Considerando la estructura `List` final que guarda K antenas ($K \leq N$), el espacio global es **$O(N)$** .

2.4 Código Fuente en Java

```
1 import java.util.ArrayList;
2 import java.util.Arrays;
3 import java.util.List;
4
5
6 public class CoberturaAntenas {
7
8     static void print(String mensaje) {
9         System.out.println(mensaje);
10    }
11
12    static List<Integer> colocarAntenas(int[] casas, int R) {
13        Arrays.sort(casas);
14
15        List<Integer> antenas = new ArrayList<>();
16        int i = 0;
17
18        while (i < casas.length) {
19            int posAntena = casas[i] + R;
20            antenas.add(posAntena);
21
22            while (i < casas.length && casas[i] <= posAntena + R) {
23                i++;
24            }
25        }
26
27        return antenas;
28    }
29
30    static void resolver(int[] casas, int R) {
31        print("== Cobertura de Antenas ==");
32        print("Casas: " + Arrays.toString(casas));
33        print("Cobertura R: " + R);
34        print("");
35
36        List<Integer> antenas = colocarAntenas(casas, R);
37
38        print("Antenas colocadas en:");
39        for (int pos : antenas) {
40            print(" Posicion: " + pos
41                + " → cubre [" + (pos - R) + ", " + (pos + R) + "]);
42        }
43        print("");
44        print("Cantidad total de antenas: " + antenas.size());
45    }
46
47    public static void main(String[] args) {
48
49        // Ejemplo 1
50        int[] casas1 = {1, 2, 7, 11, 20, 21, 30};
51        resolver(casas1, 5);
52
53        print("");
```

```

54     print("-----");
55     print("");
56
57     // Ejemplo 2
58     int[] casas2 = {2, 4, 8, 15, 18, 22};
59     resolver(casas2, 3);
60 }
61 }

```

Listing 2: Implementación algorítmica de Cobertura de Antenas

2.5 Ejemplos de Ejecución y Trazado

Ejemplo 1: Casas = [1, 2, 7, 11, 20, 21, 30]. Radio $R = 5$.

Iteración	Casa sin Cobertura	Posición Antena ($x + R$)	Alcance	Casas Cubiertas
1	Casa en $x = 1$	6	[1, 11]	{1, 2, 7, 11}
2	Casa en $x = 20$	25	[20, 30]	{20, 21, 30}

Cuadro 3: Trazado del Ejemplo 1. Total de antenas: **2** (Ubicadas en 6 y 25).

Ejemplo 2: Casas = [2, 4, 8, 15, 18, 22]. Radio $R = 3$.

Iteración	Casa sin Cobertura	Posición Antena ($x + R$)	Alcance	Casas Cubiertas
1	Casa en $x = 2$	5	[2, 8]	{2, 4, 8}
2	Casa en $x = 15$	18	[15, 21]	{15, 18}
3	Casa en $x = 22$	25	[22, 28]	{22}

Cuadro 4: Trazado del Ejemplo 2. Total de antenas: **3** (Ubicadas en 5, 18 y 25).